

Herramientas de Documentación

Oretania



Participantes: Raúl Lumbreras Delegido, Francisco Manuel Vigil Ruiz, Álvaro Colmenero Rodríguez y Alejandro Medel Martínez

Centro: I.E.S. Oretania

Curso: 2025-2026

Índice

1. Introducción y propósito
2. Stack de herramientas utilizadas
3. Flujo de trabajo
4. Justificación de decisiones
5. Guía de reproducción

1. Introducción y propósito

Este documento describe la infraestructura y el flujo de trabajo empleados para generar la documentación técnica del proyecto **OretanIA**. El objetivo era producir documentos profesionales de forma ágil, manteniendo coherencia visual entre todos los manuales y facilitando su actualización futura.

Para ello se combinaron dos herramientas principales: **Claude** (Anthropic) como motor de generación y estructuración de contenido, y **PhpStorm** como entorno centralizado de edición y exportación. Esta combinación permitió reducir significativamente los tiempos de redacción manual sin sacrificar la calidad técnica ni la precisión del contenido.

2. Stack de herramientas utilizadas

Claude (Anthropic)

Claude es el asistente de inteligencia artificial desarrollado por Anthropic que se utilizó como motor principal de generación de contenido. Su función fue estructurar los borradores técnicos, redactar las descripciones de cada sección y mantener la coherencia entre los distintos manuales.

Se le proporcionó acceso directo al código fuente del proyecto (controladores PHP, scripts Python, entidades Doctrine y configuraciones) para que el contenido generado fuera preciso y estuviera basado en la implementación real, no en suposiciones.

PhpStorm

PhpStorm es el entorno de desarrollo integrado (IDE) de JetBrains utilizado durante todo el desarrollo del proyecto. Además de servir como editor de código, se empleó como entorno centralizado para la edición y revisión de los archivos de documentación en formato Markdown (`.md`).

Su plugin de previsualización Markdown permitió visualizar el resultado en tiempo real y, mediante la funcionalidad de exportación a PDF integrada, generar los documentos finales directamente desde el IDE sin herramientas externas adicionales.

Markdown (`.md`)

Se eligió Markdown como formato de origen para toda la documentación por su simplicidad, legibilidad en texto plano y compatibilidad con múltiples exportadores. El uso de etiquetas HTML embebidas (`<div>` , `<style>`) permitió controlar el layout del PDF (saltos de página, tamaños de fuente, alineación) manteniendo el archivo editable en cualquier editor de texto.

3. Flujo de trabajo

El proceso seguido para generar cada manual fue el siguiente:

1. Extracción de contexto

Se proporcionó a Claude el código fuente del proyecto: controladores, entidades, scripts Python y archivos de configuración. Esto garantizó que la documentación reflejara el comportamiento real de la aplicación.

2. Generación del borrador

Claude estructuró y redactó el contenido de cada sección según los requisitos del manual correspondiente (usuario final, administrador técnico o despliegue). Los borradores iniciales se generaron en formato Markdown directamente.

3. Revisión y refinamiento

Los borradores se abrieron en PhpStorm, donde el equipo revisó la precisión técnica, corrigió detalles específicos del proyecto y ajustó el formato visual. Se utilizaron etiquetas HTML para controlar saltos de página, tamaños de fuente en tablas grandes y la portada de cada documento.

4. Exportación a PDF

Desde PhpStorm, con el plugin Markdown instalado, se exportó cada archivo `.md` a PDF mediante la opción *Export to PDF*. Esto generó los documentos finales con el formato visual definido en el propio Markdown.

4. Justificación de decisiones

Eficiencia

La generación asistida por IA redujo drásticamente el tiempo de redacción. En lugar de escribir cada sección desde cero, el equipo pudo centrarse en revisar y corregir contenido ya estructurado, lo que aceleró el proceso sin comprometer la calidad técnica.

Precisión

Al proporcionar a Claude el código fuente real del proyecto, el contenido generado estuvo basado en la implementación efectiva y no en descripciones genéricas. Tablas de rutas, esquemas de base de datos y configuraciones reflejan exactamente el estado del proyecto.

Consistencia visual

El uso de una plantilla Markdown común para todos los manuales (misma portada, mismo índice ampliado, mismos saltos de página) garantiza que todos los PDFs tengan una apariencia uniforme y profesional.

Mantenibilidad

Al tener los fuentes en formato Markdown dentro del propio repositorio del proyecto, cualquier miembro del equipo puede actualizar la documentación en el futuro editando el `.md` correspondiente y volviendo a exportar el PDF, sin depender de herramientas externas ni licencias adicionales.

5. Guía de reproducción

Si en el futuro se necesita actualizar o regenerar alguno de los manuales, los pasos a seguir son:

Requisitos

- **PhpStorm** con el plugin **Markdown** instalado y activado.
- Acceso al repositorio del proyecto: github.com/Alvaro-777/Proyecto

Pasos

1. Abrir el archivo Markdown

Los manuales se encuentran en la carpeta `OretanIA/docs/` :

```
docs/  
├─ manual_usuario.md  
├─ manual_administrador.md  
├─ manual_despliegue.md  
└─ herramientas_documentacion.md
```

2. Editar el contenido

Modificar el archivo `.md` correspondiente en PhpStorm. La previsualización en tiempo real (`Editor` → `Split Editor Right`) permite ver el resultado mientras se edita.

3. Exportar a PDF

Con el archivo `.md` abierto en PhpStorm:

1. Menú superior → `File` → `Export to PDF`
2. O bien clic derecho en el editor → `Export Markdown to PDF`
3. Seleccionar la ruta de salida y confirmar.

La imagen de portada (`img/logoDocs.png`) debe estar en la carpeta `docs/img/` para que aparezca correctamente en el PDF.

4. Verificar el resultado

Revisar que los saltos de página sean correctos, que las tablas no se corten y que la portada muestre la imagen. Si algún bloque se parte entre páginas, añadir `<div style="page-break-inside: avoid;">` alrededor del elemento problemático.